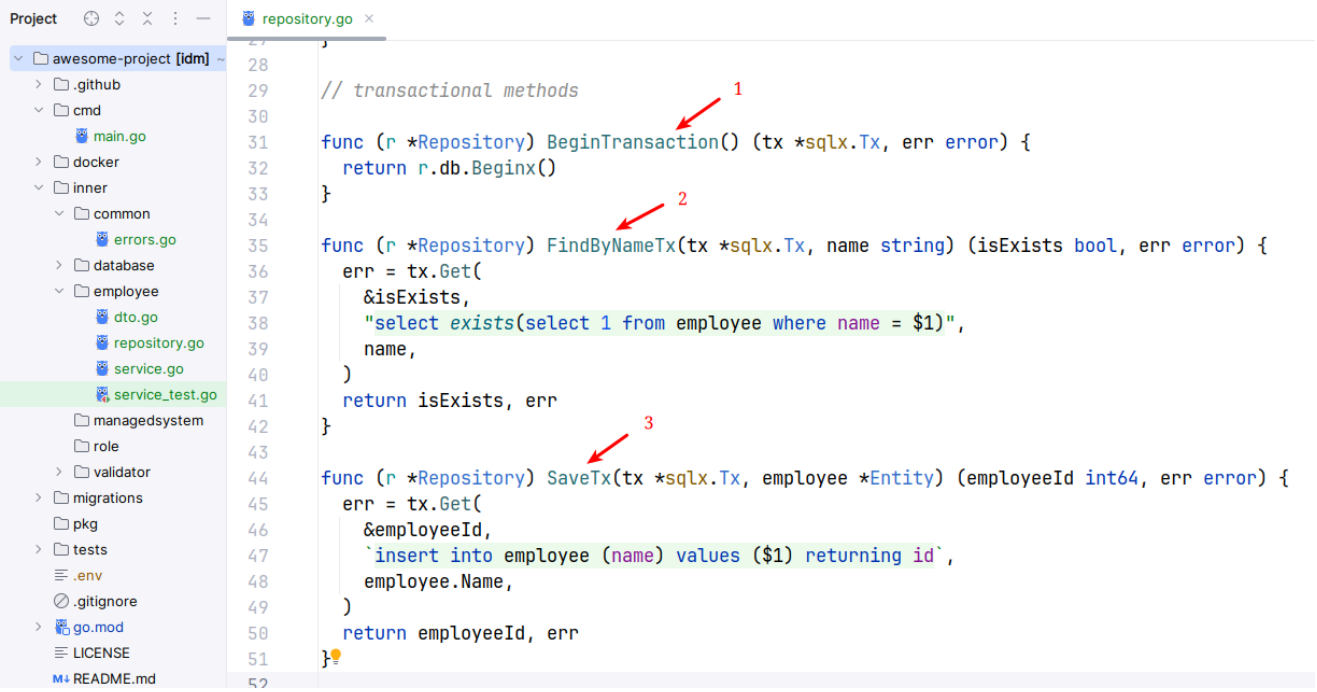
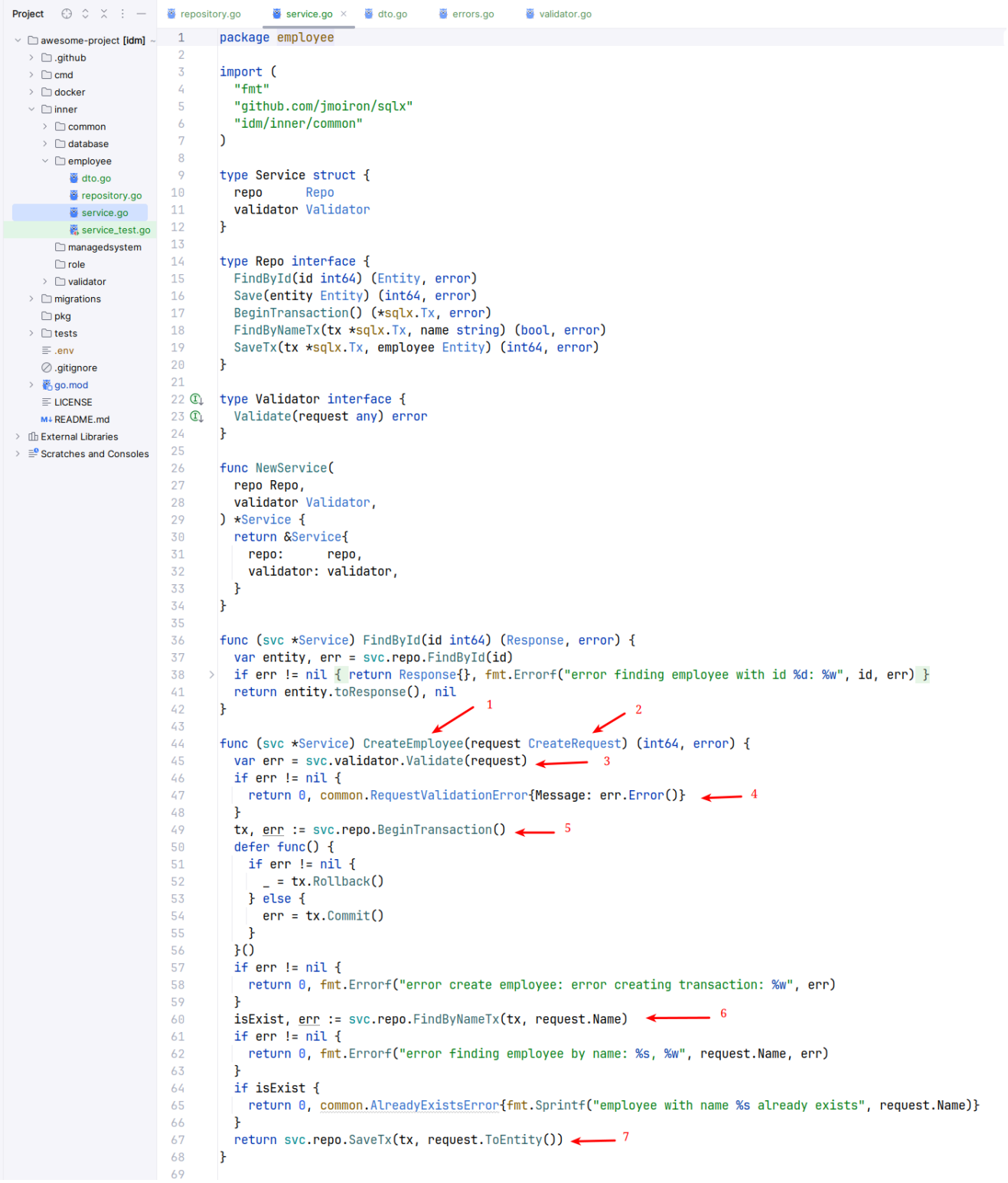
Лекция 18. Слой API. Хендлеры

Хендлеры (контроллеры, роуты) - это функции, которые соотносят URI веб-запроса с функциями бизнес-логики, которые будут их обрабатывать.
На нашем курсе, для создания веб-сервиса мы будем использовать пакет github.com/gofiber/fiber
Давайте, разберём, как мы могли бы реализовать API (эндпоинт) для создания в базе данных нового работника.
С прошлых уроков у вас в репозитории уже должны быть реализованы транзакционные методы:



(1) - создания новой транзакции
(2) - поиска работника по имени
(3) - сохранения нового работника
В сервисе для этих целей нами был создан метод:



(1) - метод для создания работника

(2) - принимает структуру запроса на создание работника

(3) - валидирует структуру запроса (про валидатор расскажу дальше)

(4) - возвращает кастомную ошибку в случае, если запрос не прошёл валидацию (про кастомные ошибки - дальше)

(5) - запрашивает у репозитория транзакцию

(6) - в рамках транзакции проверяет наличие в базе данных работника с таким же именем

(7) - в случае отсутствия работника с таким же именем -

вызывает метод репозитория, который должен будет создать нового работника и возвращает результат.

Структура запроса на создание нового работника:

```go
27
28  type CreateRequest struct {
29    Name string `json:"name" validate:"required,min=2,max=155"`
30  }
31
32  func (req *CreateRequest) ToEntity() Entity {
33    return Entity{Name: req.Name}
34  }
35
```

Содержит одно поле - имя работника, отмеченное тегами для анмаршалера (1) и валидатора (2).

Структура валидатора:

Инкасулирует работу с валидатором из пакета github.com/go-playground/validator/v10 (1). Имеет единственный метод для валидации (2).

Структуры кастомных ошибок:

```go
package common

type RequestValidationError struct {
  Message string
}

func (err RequestValidationError) Error() string { return err.Message }

type AlreadyExistsError struct {
  Message string
}

func (err AlreadyExistsError) Error() string {
  return err.Message
}
```

Структура нашего контроллера, который будет инкапсулировать все хендлеры пакета employee:

```go
package employee

import (
  "errors"
  "github.com/gofiber/fiber"
  "idm/inner/common"
)

type Controller struct {
  app             *fiber.App
  employeeService Svc
}

type Svc interface {
  FindById(id int64) (Response, error)
  CreateEmployee(request CreateRequest) (int64, error)
}

func NewController(app *fiber.App, employeeService Svc) *Controller {
  return &Controller{
    app:             app,
    employeeService: employeeService,
  }
}

func (c *Controller) RegisterRoutes() {
  api := c.app.Group("/api")
  api.Post("/v1/employees", c.CreateEmployee)
}

func (c *Controller) CreateEmployee(ctx *fiber.Ctx) {
  var request CreateRequest
  if err := ctx.BodyParser(&request); err != nil {
    _ = common.ErrResponse(ctx, fiber.StatusBadRequest, err.Error())
    return
  }
  var newEmployeeId, err = c.employeeService.CreateEmployee(request)
  if err != nil {
    switch {
    case errors.As(err, &common.RequestValidationError{}) || errors.As(err, &common.AlreadyExistsError{}):
      _ = common.ErrResponse(ctx, fiber.StatusBadRequest, err.Error())
    default:
      _ = common.ErrResponse(ctx, fiber.StatusInternalServerError, err.Error())
    }
    return
  }
  if err = common.OkResponse(ctx, newEmployeeId); err != nil {
    _ = common.ErrResponse(ctx, fiber.StatusInternalServerError, "error returning created employee id")
    return
  }
}
```

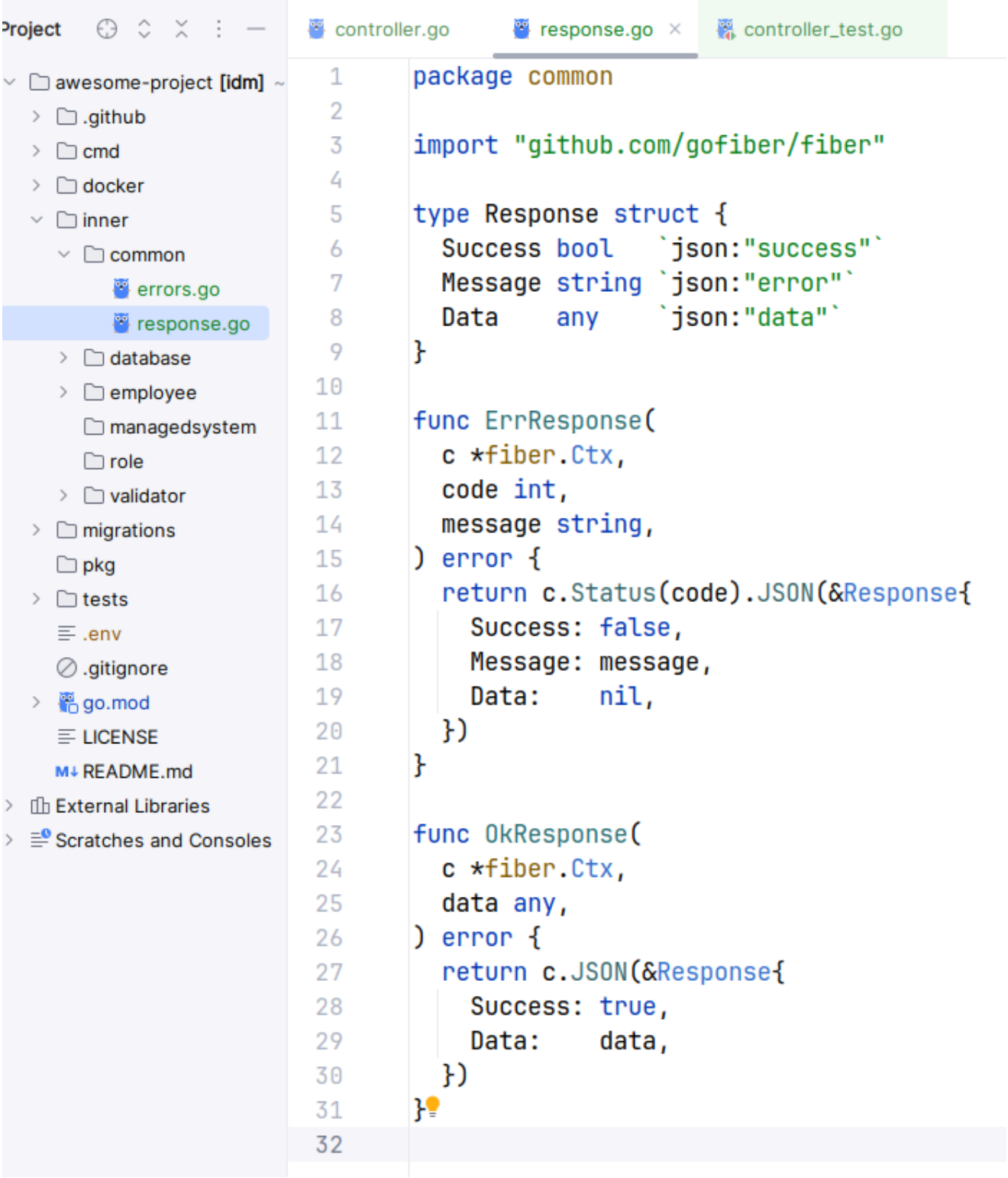
(1) - контроллер - этим термином обозначают компонент, который маршрутизирует веб-запросы на функции-хендлеры

(2) - интерфейс сервиса employee.Service

(3) - функция-конструктор

(4) - функция для регистрации маршрутов
(5) - создаём группу маршрутов "/api"
(6) - добавляем в группу наш первый маршрут "/v1/employees", который мапится на функцию-хендлер нашего контроллера c.CreateEmployee (7)
(8) - анмаршалим JSON body запроса в структуру CreateRequest
(9) - вызываем метод CreateEmployee сервиса employee.Service
(10) - если сервис возвращает ошибку RequestValidationError или AlreadyExistsError (11), то мы возвращаем ответ с кодом 400 (BadRequest) (12)
(13) - если сервис возвращает другую ошибку, то мы возвращаем ответ с кодом 500 (InternalServerError)
(14) - функция ErrorResponse() направляет ответ в случае ошибки
(15) - функция ErrorResponse() направляет ответ в случае успеха
На следующем скриншоте представлена общая структура ответа на все веб-запросы к группе "/api". И две функции, формирующие ответы в случае успешной и неуспешной обработки

веб-запроса.



Задачи на сегодня:
Написать в своём проекте:

1. валидатор запросов
2. структуры запросов с тегами для валидации
3. тесты валидации всех структур запросов. Случаи, которые нужно протестировать:

- все поля заполнены корректно

- поочереди некорректно заполнять одно из полей и проверять, что валидатор правильно определит некорректно заполненное поле

4. кастомные ошибки
5. контроллеры для всех пакетов
6. в контроллерах написать хендлеры для мапинга веб-запросов на все методы сервисов. Предусмотреть возврат различных HTTP статус-кодов для различных ситуаций.